

ATVTI — CCIS Internship Program 2026

Day 1 Study & Build Guide

First Half: How the Web Works | Second Half: Your First HTML Page

Day	Day 1 — Foundations
First Half Goal	Understand how the web works and set up your tools
Second Half Goal	Write your first HTML page and push it to GitHub
Key Tools	VS Code, Node.js, Git, GitHub
Deliverable	index.html pushed to your own branch on the team repo

FIRST HALF

How the Web Works

1. How a Web Page Reaches Your Screen

Every time you open a website, a lot happens behind the scenes in less than a second. Here is that journey, step by step.

1.1 The Client–Server Model

The internet is built on two roles:

- Client — your browser (Chrome, Firefox, Edge). It asks for things.
- Server — a computer somewhere in the world that stores files and responds to requests.

Real-world analogy

Think of a restaurant. You (the client) sit at a table and order food.

The kitchen (the server) prepares it and sends it back to you.

The waiter carrying your order back and forth is like the HTTP protocol.

1.2 What is HTTP?

HTTP stands for HyperText Transfer Protocol. It is the agreed-upon language that browsers and servers use to talk to each other.

- When you type a URL and press Enter, your browser sends an HTTP Request to the server.
- The server reads the request, finds the right file, and sends back an HTTP Response.
- That response usually contains HTML, which your browser turns into the page you see.

HTTP Method	What It Does
GET	Fetch a page or resource (most common — used when you visit a URL)
POST	Send data to the server (e.g., submitting a login form)
PUT / PATCH	Update existing data on the server
DELETE	Remove a resource from the server

1.3 What is a URL?

URL stands for Uniform Resource Locator. It is the address of a resource on the internet. Every URL has the same structure:

URL breakdown — <https://atvti.edu.pk:443/courses/web-dev?id=1>

[https://](https://atvti.edu.pk:443/courses/web-dev?id=1) — Protocol (how to connect; 's' means secure / encrypted)
[atvti.edu.pk](https://atvti.edu.pk:443/courses/web-dev?id=1) — Domain name (the human-readable address of the server)
[:443](https://atvti.edu.pk:443/courses/web-dev?id=1) — Port (the 'door' on the server; 443 is the default for HTTPS)
[/courses/web-dev](https://atvti.edu.pk:443/courses/web-dev?id=1) — Path (which specific page/file you want)
[?id=1](https://atvti.edu.pk:443/courses/web-dev?id=1) — Query string (extra parameters sent to the page)

1.4 What is a Domain and a Port?

- Domain — a name like google.com or atvti.edu.pk. Behind the scenes it maps to a numbered IP address (e.g. 142.250.180.14). DNS (Domain Name System) does this translation, like a phone book for the internet.
- Port — a number (0–65535) that identifies a specific service running on a server. Port 80 = plain HTTP. Port 443 = HTTPS. Port 3000 = a common default for Node.js development servers.

Why ports matter for you

When you run a Node.js server locally, it will listen on a port such as 3000. You open it in the browser at: <http://localhost:3000>
 'localhost' means your own computer. Port 3000 is where your server is listening.

1.5 What the Server Returns — HTML

When a browser sends a GET request for a page, the server most often replies with an HTML file. HTML (HyperText Markup Language) is the skeleton of every web page. The browser reads it and draws the page on your screen.

- HTML provides structure (headings, paragraphs, buttons, images).
- CSS (Cascading Style Sheets) adds visual styling (colours, fonts, spacing).
- JavaScript adds behaviour (responding to clicks, loading data, animations).

2. Setting Up Your Tools

You need three core tools installed before you can write and share code professionally.

2.1 VS Code — Your Code Editor

VS Code (Visual Studio Code) is a free, lightweight code editor made by Microsoft. It is the industry standard for web development. It highlights your code in colour, catches mistakes, and has thousands of extensions.

Installation steps:

1. Go to code.visualstudio.com and download the installer for your OS.
2. Run the installer and accept all default options.
3. After opening VS Code, press `Ctrl + Shift + X` to open Extensions.
4. Search for and install: Prettier (code formatter), Live Server (preview HTML instantly), GitLens (Git inside VS Code).

2.2 Node.js — JavaScript on the Server

Node.js lets you run JavaScript outside the browser, on your own computer or a server. Even though today you are only writing HTML, you will need Node.js installed for later weeks when you build a backend with Express.

Installation steps:

5. Go to nodejs.org and download the LTS (Long Term Support) version.
6. Run the installer with default settings. It also installs npm (Node Package Manager) automatically.
7. Open a terminal and verify by typing:

```
node --version  
npm --version
```

Both commands should print a version number. If they do, you are ready.

2.3 Git — Version Control

Git is a tool that tracks every change you make to your code. It is like an undo history for your entire project, and it allows a team to work on the same codebase without overwriting each other's work.

Key concepts:

- Repository (repo) — a folder that Git is tracking. The team project lives in one shared repo.
- Commit — a saved snapshot of your work at a point in time, with a message describing what changed.
- Branch — a separate line of development. You will create your own branch so your changes don't break the main codebase.
- Push — upload your commits from your local machine to GitHub.
- Pull — download the latest changes from GitHub to your machine.

Installation steps:

8. Go to git-scm.com and download Git for your OS.
9. Run the installer with default settings.
10. Open a terminal and configure your identity (Git will attach this to your commits):

```
git config --global user.name "Your Name"  
git config --global user.email "youremail@example.com"
```

2.4 GitHub — Git in the Cloud

GitHub is a website that stores your Git repositories online. It lets the whole team see each other's code, review changes, and collaborate.

Setup steps:

11. Go to github.com and create a free account using your email.
12. Choose a professional username (e.g., rafay-atvti or firstname-lastname).
13. Verify your email address via the confirmation link GitHub sends.
14. Your instructor will add you as a collaborator to the team repository. Accept the invitation from your email.

Second Half Session

Clone the repo. Create your branch. Write your first page. Push to GitHub.

SECOND HALF

Your First HTML Page on GitHub

3. Step-by-Step: Clone → Branch → Build → Push

Step 1 — Clone the Team Repository

Cloning copies the team repo from GitHub onto your local machine. You only do this once per machine.

15. Open a terminal (Git Bash on Windows, Terminal on Mac/Linux).

16. Navigate to where you want to keep projects. For example:

```
cd Documents
```

17. Clone the repo (your instructor will provide the actual URL):

```
git clone https://github.com/atvti-ccis/team-website.git
```

18. Move into the cloned folder:

```
cd team-website
```

19. Open the entire folder in VS Code:

```
code .
```

What just happened?

Git downloaded all the files from GitHub to your computer.

You are now on the 'main' branch, which is the official copy.

The next step creates your own branch so you can work independently.

Step 2 — Create Your Own Branch

A branch is your personal workspace. Changes on your branch do not affect the main branch until they are reviewed and merged.

20. In the terminal (inside the team-website folder), create and switch to your branch:

```
git checkout -b yourname-day1
```

Replace yourname with your actual first name in lowercase. For example: ahmed-day1 or sana-day1.

21. Verify you are on the right branch:

```
git branch
```

The branch with an asterisk (*) next to it is your current branch.

Step 3 — Understand the Project Structure

Before writing any HTML, look at the folder structure your instructor has set up:

```
team-website/  
  index.html      ← the main homepage  
  assets/
```

css/	← stylesheets go here
images/	← image files go here
pages/	← additional pages
README.md	← project description

Today you are only working on index.html. Your task is to write the page structure — no CSS styling required yet.

Step 4 — Write index.html

Open index.html in VS Code and type the following (do not copy-paste — typing it builds muscle memory and you will understand each line better):

What each HTML tag does

<!DOCTYPE html>	Tells the browser this is HTML5 (the modern standard).
<html lang="en">	The root element wrapping the entire page.
<head>	Contains metadata not shown on the page (title, links to CSS).
<meta charset>	Ensures characters like Urdu or Arabic display correctly.
<meta name=viewport>	Makes the page responsive on mobile screens.
<title>	Text shown in the browser tab.
<body>	Everything visible on the page goes inside here.
<nav>	Navigation bar — semantic tag for menus.
<header>	Introductory content of a section or page.
<h1>	The most important heading on the page (use only one per page).
<p>	A paragraph of text.
<footer>	Bottom section of the page — contact, links, copyright.

Write the following structure in your index.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>ATVTI — Center for Computing and Intelligent Systems</title>
</head>
<body>

  <!-- NAVBAR -->
  <nav>
    
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">Courses</a></li>
      <li><a href="#">About</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
```

```

</nav>

<!-- HERO SECTION -->
<header>
  <h1>Welcome to ATVTI - CCIS</h1>
  <p>Building the next generation of technology professionals in
Pakistan.</p>
</header>

<!-- FOOTER -->
<footer>
  <p>&copy; 2026 ATVTI. All rights reserved.</p>
</footer>

</body>
</html>

```

Step 5 — Preview Your Page

Before committing anything, verify that your HTML looks correct in the browser.

22. In VS Code, right-click index.html in the file explorer on the left.
23. Select 'Open with Live Server'.
24. A browser window will open showing your page. You should see the heading, paragraph text, and nav items.
25. If nothing appears, check that your file is saved (Ctrl + S).

Step 6 — Commit Your Work

Now save your progress to Git. Every commit tells a story about what you built. Write clear, descriptive commit messages — they are part of your professional record.

26. In the terminal, stage all your changes:

```
git add .
```
27. Create a commit with a message:

```
git commit -m "Day 1: Add ATVTI navbar, hero section, and footer"
```

Good vs bad commit messages

BAD: 'git commit -m "changes"'

BAD: 'git commit -m "done"'

GOOD: 'git commit -m "Day 1: Add navbar with logo placeholder and nav links"'

A good message says WHAT you did, not that you did something.

Step 7 — Push to GitHub

Pushing uploads your local commits to GitHub so your instructor and teammates can see your work.

28. Push your branch to GitHub:

```
git push origin yourname-day1
```
29. Open GitHub in the browser and navigate to the team repository.

30. You should see a banner saying your branch was recently pushed. Click 'Compare & pull request' — you do not need to open the PR today, but check that your branch is visible.

What 'origin' means

'origin' is the default name Git gives to the remote repository (GitHub).

When you cloned the repo, Git automatically named that remote connection 'origin'.

'git push origin yourname-day1' means: push my local branch to GitHub.

4. End-of-Day Checklist

Before you leave, make sure all of the following are done:

☐	VS Code is installed and the three extensions (Prettier, Live Server, GitLens) are active.
☐	Node.js is installed — <code>node --version</code> prints a version number.
☐	Git is installed and configured with your name and email.
☐	You have a GitHub account and accepted the team repo invitation.
☐	You have cloned the team repository to your local machine.
☐	You created your own branch (yourname-day1) and are working on it.
☐	index.html contains a navbar, a hero section with an h1, and a footer.
☐	Your page loads correctly in the browser via Live Server.
☐	You made at least one commit with a descriptive message.
☐	You pushed your branch to GitHub and can see it on the GitHub website.

5. Key Concepts Summary

Term	Plain-English Definition
HTTP	The rules that govern how browsers and servers communicate.
URL	The full address of a web resource, including protocol, domain, port, and path.
Domain	The human-readable name of a server (e.g., atvti.edu.pk) that maps to an IP address.
Port	A numbered door on a server that a specific service listens on (80, 443, 3000, etc.).
HTML	The markup language used to structure the content of a web page.

Git	A version-control system that tracks changes to code over time.
Repository	A project folder that Git is tracking, either locally or on GitHub.
Branch	An independent line of development that keeps your changes separate from the main code.
Commit	A saved snapshot of changes with a descriptive message.
Push	Uploading local commits to the remote repository on GitHub.
Clone	Downloading a complete copy of a GitHub repository to your local machine.
localhost	The address your browser uses to reach a server running on your own computer.

6. Common Mistakes to Avoid

- Working directly on the main branch — always create and switch to your own branch first.
- Vague commit messages like 'fix' or 'update' — describe exactly what you changed.
- Not saving your file before previewing — always press Ctrl + S before checking Live Server.
- Forgetting to push — committing saves locally; only pushing shares your work with the team.
- Unclosed HTML tags — every opening tag like <nav> must have a matching </nav>.
- Wrong indentation — consistent indentation (2 spaces) makes HTML much easier to read and debug.

If you are stuck, ask — not after an hour of guessing. Good engineers ask early.

ATVTI — Centre for Computing and Intelligent Systems (CCIS) | Internship Program 2026